

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly being coupled with graph-structured data for tasks such as node classification, a setting historically dominated by Graph Neural Networks (GNNs). Although this integration exploits rich relational information and has yielded substantial task-performance gains, the adversarial robustness of graph-aware LLMs remains largely unexamined. We provide an initial systematic study of this vulnerability landscape by adapting established adversarial attack methods for graph-based models, including poisoning attacks executed at training time and evasion attacks executed at test time, to two representative graph-aware LLMs, LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). Beyond these transferred attacks, we identify a previously unreported attack surface in LLAGA, wherein an adversary can inject malicious nodes as placeholders into the node sequence template and thereby substantially degrade model performance. Our analysis shows that particular graph-encoding design choices can amplify attack effectiveness: (1) the node sequence template in LLAGA increases susceptibility; (2) the GNN encoder used in GRAPH-PROMPTER exhibits comparatively greater robustness; and (3) both paradigms remain vulnerable to imperceptible feature perturbations. Finally, we propose GALGUARD, an end-to-end defense framework that combines an LLM-based feature correction module for mitigating feature-level perturbations with adapted GNN defenses for protecting against structural attacks¹

1 Introduction

Graphs, whose nodes and edges encode entities and their relations, provide a natural representational substrate for complex systems across social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), which are explicitly designed for graph-structured inputs, have therefore become central to tasks such as node classification and link prediction. More recently, however, the empirical success of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna

(Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), whose training is grounded in massive textual corpora, has motivated substantial interest in their capacity to augment or replace conventional graph-learning pipelines. This coupling of graph data with LLMs, often termed *graph-aware LLMs* (Xie et al. 2023), has consequently opened a rapidly expanding research direction in which LLMs are increasingly used for problems previously addressed primarily by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Indeed, recent evidence indicates that graph-aware LLMs can outperform GNNs on several graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), thereby positioning them as powerful alternatives rather than merely auxiliary components.

Despite their increasing prominence, the susceptibility of graph-aware LLMs to adversarial manipulation remains insufficiently understood. Existing adversarial attacks against GNNs, including poisoning attacks at training time and evasion attacks at test time (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), commonly assume a black-box setting in which the adversary lacks access to the model’s internal architecture and parameters. Such attacks undermine *model integrity* by intentionally degrading predictive performance through subtle perturbations to graph structure or node features. Whereas the GNN robustness literature has concentrated primarily on structural perturbations, the textual interface through which graph-aware LLMs ingest node attributes makes imperceptible feature-level manipulation a particularly salient, yet comparatively neglected, source of risk.

Motivated by the migration from GNNs toward LLM-centered graph pipelines, the black-box nature of canonical graph adversarial attacks, and the emergence of feature perturbations as a potentially distinctive vulnerability, we ask whether *adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm*. Answering this question requires attention to how graph information is incorporated into the language model, because graph-aware LLM performance *highly depends* on the fidelity of this integration. Although processing node features is relatively direct given the textual competence of LLMs, representing graph topology is more difficult and demands mechanisms capable of preserving relational information

¹Our code will be released upon acceptance.

(Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). We therefore first categorize existing graph-aware methods into two broad classes according to how they encode graphs for LLMs: approaches that express graph data as *textual descriptions*, thereby converting graph information into natural language, and approaches that employ *learned projectors* to bridge the representational gap between graph and text domains. Because conventional text-based adversarial attacks are generally sufficient to compromise LLMs operating on textual graph descriptions, our empirical investigation focuses on learned-projector methods, using two computationally efficient representative models, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which rely on frozen pre-trained LLMs.

Our analysis demonstrates that seemingly innocuous design choices in graph encoding can unexpectedly magnify adversarial effectiveness. In particular, we uncover a new attack surface in LLAGA, where an adversary can substantially degrade performance by injecting malicious nodes as placeholders inside the node sequence template. We further introduce an end-to-end graph-aware LLM defense, GALGUARD, which integrates an LLM-based feature corrector with adapted GNN-based structural defenses to provide more comprehensive protection. To our knowledge, this work is the first systematic investigation of adversarial vulnerability in graph-aware LLMs. The contributions of the paper are summarized as follows:

- We adapt representative GNN adversarial attacks, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to assess the transferability of poisoning and evasion attacks to graph-aware LLMs.
- We develop a graph-encoding taxonomy for graph-aware LLMs, enabling a clearer analysis of how distinct integration strategies expose LLMs to different structural and feature-level vulnerabilities.
- We identify a new attack surface in LLAGA’s node sequence template and design node sequence template injection attacks that exploit placeholder positions to corrupt the model’s graph-derived representations.
- We demonstrate that imperceptible feature perturbations, which are often less consequential for traditional GNNs, can be highly effective against graph-aware LLMs and can outperform structural attacks in several settings.
- We propose GALGUARD, an end-to-end defense framework that combines LLM-based feature correction with adapted GNN-inspired structural defenses to mitigate both feature and structural perturbations.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs can be organized into two principal families: *textual descriptions of graphs*, in which graph information is verbalized as natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured representations that

an LLM can process directly. We briefly characterize both families in order to locate the models studied in this paper within the broader design space. Figure 5 illustrates these two approaches, while Appendix A.1 provides a fuller taxonomy and Appendix A.2 reviews related work on GNN attacks.

Graph Representation as a Textual Description. This approach translates graph data into natural-language descriptions, typically coupled with task-specific instructions that guide LLM querying (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multi-modal contexts, Liu et al. (2023) and Su et al. (2022) associate molecular graphs with textual descriptions, whereas Huang et al. (2023a) examine how the inclusion of graph structure in prompts affects prediction. Their findings indicate that LLMs tend to interpret graph-related prompts as contextual paragraphs rather than as explicit graph structures, thereby emphasizing the importance of prompt design in subsequent studies (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Additional work uses LLMs to enhance node features, augment graph structures, function as direct predictors, or act as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) propose a textual-graph fine-tuning method that combines LLMs and GNNs, while Duan et al. (2023) introduce an effective strategy for improving textual graph learning. Although these methods offer a comparatively direct route for applying LLMs to graph tasks, they often inadequately capture structural intricacy, especially in large graphs, and remain heavily dependent on prompt engineering.

Graph Representation as a Learned Projector. Learned-projector methods address the limitations of purely textual graph descriptions by transforming graph data into embeddings or graph vectors that can be consumed more directly by an LLM. This family includes LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), as well as approaches that introduce GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other variants employ graph neural prompts for knowledge-graph encoding (Tian et al. 2024), construct prompt-based node feature extractors using a GNN adapter (Huang et al. 2023b), or compress graph information into fixed-length prefixes with graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We focus on projector-based methods that use *frozen pre-trained LLMs*, because this design substantially reduces computational cost while avoiding full LLM retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our investigation uses two representative models: LLAGA (Chen et al. 2024), which employs node-level templates with a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which combines a GNN with a linear projector. We exclude textual-description methods because they face scalability constraints and, more importantly for this study, do not introduce the novel graph-specific attack surfaces that arise when structural information is em-

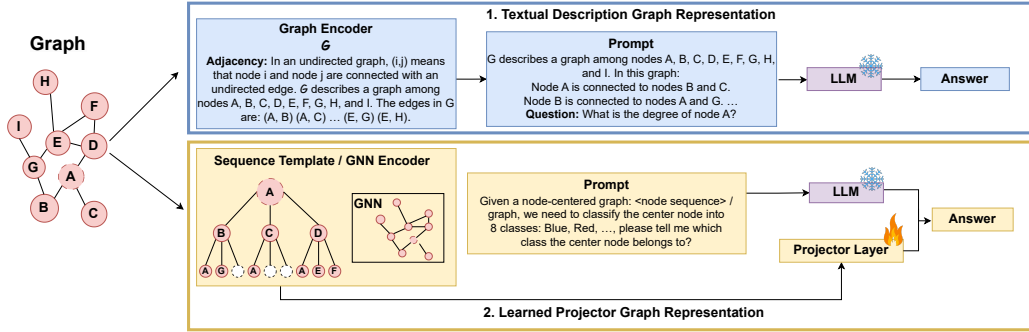


Figure 1: **Graph encoding-based adaptations of LLMs.** Under textual-description representations, graph information is rendered as natural-language text (top). Under learned-projector representations, graph information is encoded either through node templates that convert topology into sequences or through GNN-based encoders (bottom). In both settings, the LLM remains frozen, and only the projector mapping graph information into token space is trained.

bedded through learned graph-to-token mechanisms.

3 Threat Model

We formulate our threat model by extending established assumptions from adversarial graph learning to the graph-aware LLM setting (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). In a social-network deployment, for example, an adversary may control only a small number of bot accounts, a limited-access regime that matches our setting in which the attacker manipulates a small subset of nodes to evade detection or influence the classification of legitimate users. Analogously, in Wikipedia-like environments, sparsely connected hoax articles can be strategically linked to genuine articles in order to induce misclassification of legitimate content categories. This transferability is particularly concerning because graph-aware LLMs may be deployed in consequential domains such as recommendation systems, social media analysis, and healthcare, potentially replacing conventional GNNs. Following prior graph-adversarial work, we therefore specify the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—and then define the attacker’s objective.

Model Access. The attacker operates in a black-box setting, possessing no knowledge of the model architecture or parameters and influencing the model only indirectly through the training or test data.

Adversarial Capabilities. Consistent with existing poisoning and evasion attacks, the attacker has complete access to the training data in the poisoning setting and to the inference data in the evasion setting. Subject to a predefined perturbation budget and unnoticeability constraints intended to avoid detection, the attacker can modify or inject data.

Attack Strategies. The attacker uses two primary strategies: feature manipulation, which alters node features, and structure manipulation, which adds or removes edges.

Attacker’s Goal. The attacker’s objective is to compromise model integrity by degrading overall predictive performance, specifically through an increased misclassification rate in the

classification model.

4 Experimental Setup

Dataset. We evaluate on three widely used text-attributed graph benchmarks, Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016), whose variation in scale and sparsity makes them standard testbeds for models of the kind studied here (Chen et al. 2023a, 2024; Liu et al. 2024). To further examine scalability beyond these small- to medium-scale settings, we additionally include the larger ArXiv dataset (Hu et al. 2020) in a supplementary experiment (see Appendix A.5). Following (Chen et al. 2024), we use standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv; detailed dataset statistics appear in Table 1.

Table 1: Statistics of the graph datasets used in the main experiments.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We adopt *node classification* as the primary evaluation task, both because it is a canonical benchmark for graph machine learning models and because it is the dominant setting in prior work on graph adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, defined as the proportion of correctly predicted labels, as the evaluation metric. Each experiment is repeated three times, and we report the corresponding mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We next specify the graph-aware LLM architec-

Table 2: Accuracy of graph-aware LLMs LLAGA and GRAPHPROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

tures and adversarial attack methods selected for the vulnerability analysis.

Graph-Aware LLM Methods. We study two representative graph-aware LLM methods, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both methods rely on a *frozen pre-trained LLM*, which makes them computationally efficient and readily adaptable across graph tasks and LLM backbones. LLAGA uses node-level templates followed by a linear projector, whereas GRAPHPROMPTER uses a GNN-based encoder together with a linear projector; in both cases, the projector is implemented as a 2-layer multi-layer perceptron (MLP). The foundation LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We conduct the investigation using poisoning and evasion variants of two representative adversarial methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted method that searches for minimal perturbations capable of inducing misclassification for specific nodes, whereas METAATTACK performs untargeted attacks by treating graph structure as an optimizable hyperparameter. For NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019) by sequentially targeting each node and aggregating the outcomes, thereby converting the procedure into an untargeted evaluation. Across all experiments, and consistent with (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019), we perturb only 10% of the edges in order to preserve the unnoticeability constraint; additional details are provided in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to the graph-aware LLM setting. In poisoning attacks against LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector; for GRAPHPROMPTER, the perturbed graph is re-encoded with a GAT before projector retraining. In evasion attacks, by contrast, both models are trained on unperturbed data, and adversarial perturbations are introduced only at inference time.

Results. The results in Table 2 yield several observations. First, poisoning attacks exert a comparatively limited effect on graph-aware LLMs, with maximum performance degradation of 11% across the evaluated attack methods, suggesting that the learned representations and the additional contextual capacity supplied by the LLM confer a degree of resilience. Evasion attacks, however, are substantially more damaging, producing performance degradation as high as 51%. Second, across datasets and attack types, METAATTACK generally outperforms NETTACK, which is consistent with METAATTACK’s capacity to construct small but influential perturbations by optimizing graph edges as hyperparameters. Finally, denser datasets appear more vulnerable than sparser ones: for example, LLAGA incurs a 51% performance decrease on the denser Cora dataset under evasion, but only a 19% decrease on the sparser PubMed dataset.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) indicate that LLAGA is more vulnerable than GRAPHPROMPTER because its graph encoding is especially *sensitive to structural perturbations*. LLAGA incorporates graph structure through two mechanisms: a fixed-length node-level sequence for each node and a graph Laplacian embedding. Each sequence position corresponds to a specific relative structural location in the original graph, so perturbations directly alter the node embeddings subsequently processed by the LLM and thereby induce performance degradation. By contrast, GRAPHPROMPTER’s textual node embeddings remain unchanged under structural perturbation because only the GNN component is affected; consequently, the unperturbed textual features can partially compensate for the corrupted graph structure, improving robustness.

6 Node Sequence Template Injection Attacks

We identify a new attack surface in LLAGA by exploiting the neighborhood detail template used to construct node sequences; the attack is illustrated in Figure 2, and the corresponding algorithm appears in Appendix A.4. To obtain the node sequence for a given node u , LLAGA’s neighborhood detail template first builds a *fixed-shape* computational tree whose children are sampled 1-hop neighbors, and this sampling procedure is recursively applied at each subsequent

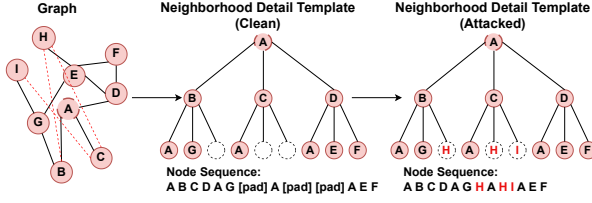


Figure 2: Illustration of node sequence template injection attacks, using Non-adjacent Injection (NI) as the displayed example. Red dashed lines denote attacker-induced edge perturbations. Under Supernode Injection (SI) and Multiple Supernode Injection (MSI), one or more supernodes are inserted into placeholder positions; here, $k = 3$.

tree level. When a node has fewer than the required number of neighbors, placeholders are inserted to preserve the fixed arity, e.g., $k = 10$ children, in accordance with LLAGA’s original design (see “Clean” in Figure 2 and lines 1-14 in the algorithm). This fixed template, while useful for standardizing graph-to-sequence conversion, creates an exploitable representational aperture.

By manipulating the graph so that malicious nodes occupy placeholder positions, an attacker can substantially degrade LLAGA’s performance without changing the overall template mechanism. We instantiate this vulnerability through three strategies. *Non-Adjacent Injection (NI)* inserts nodes that are outside the 2-hop neighborhood of a target node u (see “Attacked” in Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node, also constrained to lie outside the target node’s 2-hop neighborhood, into a placeholder position. *Multiple Supernode Injection (MSI)* extends *SI* by placing multiple distinct supernodes into placeholder positions (lines 35-43). The underlying rationale is that uninformative or structurally misleading nodes contaminate the node sequence, distort the target representation, and consequently impair projection-layer training, thereby compromising both learning and inference integrity.

Results. As shown in Table 3, LLAGA remains relatively resilient to poisoning attacks on Cora and Citeseer, yet it experiences pronounced degradation under evasion, with accuracies falling to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for *NI*, *SI*, and *MSI*, respectively.

On the larger PubMed dataset, by contrast, the model exhibits greater resilience to all injection variants, with only moderate evasion-time accuracy reductions (0.85, 0.82, and 0.76). A plausible explanation is that localized injections in smaller, more tightly connected graphs such as Cora and Citeseer exert a more concentrated influence on neighborhood reasoning, whereas the same perturbation is comparatively diluted in the much larger PubMed graph and therefore produces less severe degradation.

Relative to traditional graph adversarial methods, node injection attacks are often more effective. For example, under evasion, *MSI* yields accuracies of 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, whereas METAATTACK yields 0.44, 0.50, and 0.73 (Table 2).

Table 3: Accuracy under node sequence template injection attacks across datasets and attack variants. *NI*, *SI*, and *MSI* denote non-adjacent injection, single-supernode injection, and multiple-supernode injection, respectively, while Clean denotes model performance in the absence of attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks predominantly emphasize structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), with NETTACK (Zügner, Akbarnejad, and Günnemann 2018) as a partial exception that nevertheless does not transfer naturally to the textual domain. Because structural attacks are only moderately effective against more robust models such as GRAPH PROMPTER, and because textual features materially contribute to graph-aware LLM robustness, we investigate unnoticeable feature perturbation attacks as an additional mechanism for degrading performance.

We adopt the imperceptible attack strategies of (Boucher et al. 2022), with one key modification: perturbations are generated using a task-agnostic translation objective from English to French. The attack modifies input text so that it remains visually identical or nearly identical to a human reader while inducing an incorrect translation, thereby corrupting the semantics encoded by the model. The intuition is that an erroneous translation signals a corresponding distortion in the encoded semantic representation. We consider two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction-control characters to alter rendering order. Examples of Homoglyph and Reordering attacks appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between model outputs on perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature. **Results.** As Table 4 shows, Reordering attacks are generally more potent than Homoglyph attacks across models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attack types: poisoning attacks produce accuracy drops of 27% and 23% under Homoglyph perturbations and larger drops of 35% and 25% under Reordering perturbations, respectively. In both cases, GRAPH PROMPTER is comparatively more resilient. PubMed follows the same qualitative

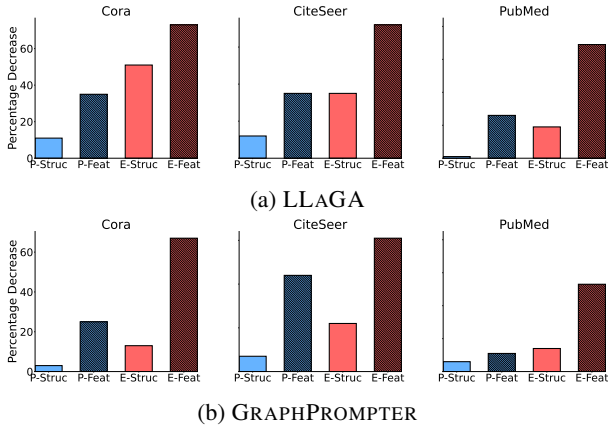


Figure 3: Performance degradation induced by structure- and feature-based adversarial attacks on graph-aware LLMs. Attack success rate is measured as the percentage decrease from original accuracy on the y-axis; P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural and feature perturbations.

pattern, although the overall degradation is less pronounced than on the smaller datasets.

Comparison with structural perturbations reveals a substantial shift in the vulnerability profile, as shown in Figure 3. Feature perturbation attacks, especially Reordering, outperform structural attacks based on METAATTACK by a large margin on both LLAGA and GRAPH PROMPTER. This pattern suggests that graph-aware LLMs rely more heavily on textual input features than on graph structure, thereby increasing their exposure to feature-level perturbations. We examine this interaction further by combining both attack types in a unified setting in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that combines feature and structural perturbations in order to construct a more effective yet still imperceptible adversarial strategy. Our hypothesis is that the joint manipulation of these two modalities will degrade model accuracy more severely than either perturbation type in isolation, while preserving inconspicuous alterations to both graph structure and node features.

For the structural component, we use METAATTACK, which modifies graph topology through subtle edge adjustments. For the feature component, we combine Homoglyph and Reordering attacks as described in the previous section. Together, these perturbations form a highly imperceptible attack that simultaneously targets the relational and textual signals on which graph-aware LLMs depend.

Results. As shown in Table 5, the unified attack, which applies structural and feature perturbations jointly, is substantially more effective than either perturbation type alone across datasets. This result underscores an important property of graph-aware LLMs: unlike traditional GNNs, which are primarily affected by structural perturbations, these models ex-

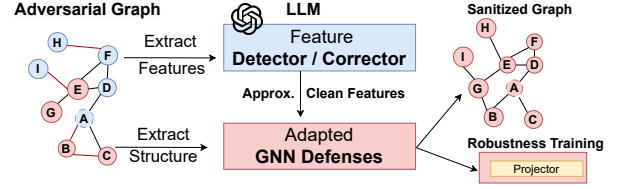


Figure 4: **Overview of the GALGUARD defense.** Perturbed node features and edges are shown in blue and red, respectively. The LLM detects anomalous perturbations in node features extracted from an adversarial graph and produces a corrected version. The refined features, together with the perturbed edges, are then passed through existing GNN defenses, which can either sanitize the graph or be adapted for robust projector training.

hibit pronounced vulnerability to feature-level manipulation. The strength of the unified attack indicates that graph-aware LLMs depend on the consistency of both graph topology and textual node features, making them especially susceptible to coordinated, inconspicuous modifications.

9 Ablation Study

9.1 Varying Feature Perturbation

To clarify the relationship between perturbation magnitude and attack success, we vary the percentage of perturbed features. Our analysis of the Reordering attack, which follows patterns similar to the Homoglyph attack, reveals a consistent trend as the perturbation percentage ranges from 1% to 10%: for all models and datasets (Cora, Citeseer, and PubMed), reducing the proportion of perturbed features predictably weakens the attack. Notably, as the perturbation level decreases, poisoning and evasion success rates tend to converge, because the adversarial signal becomes too weak to substantially affect the model regardless of whether it is introduced during training or inference. A detailed plot illustrating these trends appears in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We further evaluate whether the choice of foundation LLM influences adversarial robustness in graph-aware LLMs. Since LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPH PROMPTER uses Llama2-7B (Touvron et al. 2023), we replace LLAGA’s backbone with Llama2-7B and compare performance under poisoning and evasion attacks across all datasets. Both backbones achieve similar clean accuracy (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and exhibit comparable adversarial vulnerability; the same trend holds when swapping LLMs in GRAPH PROMPTER. These results suggest that foundation-LLM choice has minimal impact on adversarial robustness. Full results are reported in Table 2 of the Appendix.

Table 4: Accuracy under feature perturbation attacks, with the percentage decrease from clean accuracy reported in parentheses.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

10 Defense: GALGUARD

Existing GNN defenses against adversarial attacks are insufficient for graph-aware LLMs because these models are simultaneously exposed to structural and feature perturbations. Many defenses prune or reweight edges according to node-feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but this strategy presupposes access to clean features; once the features themselves are perturbed, the similarity signal becomes unreliable and the defenses fail (see Figure 3 in the Appendix). Likewise, defenses that decouple structural and feature views (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), as well as OCR-based sanitization (Boucher et al. 2022), can break down under coordinated attacks. This dual vulnerability motivates an integrated defense rather than a direct transplantation of existing GNN defenses.

To address this limitation, we propose graph-aware LLM defense (GALGUARD), an end-to-end strategy that combines two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As illustrated in Figure 4, an LLM functions as an adaptive feature-inspection mechanism that analyzes textual node attributes for anomalous patterns. It then generates approximate corrections intended to restore feature integrity, thereby reducing the effect of feature-level perturbations. This LLM-based correction provides a practical mitigation for textual feature vulnerabilities; the specific instructions and prompts are provided in Appendix A.6. For our experiments, we use GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To improve structural robustness, we use a two-stage procedure. First, we adopt a *graph purification heuristic* (Wu et al. 2019) as pre-processing: for each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and the embeddings $\phi(x_u)$ of its neighbors, removing edges whose similarity is low. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, GNNGuard’s principles are applied directly to the message-passing layers. For LLAGA, we introduce a tailored adaptation in which purification is incorporated into the construction of the computational tree used for the node sequence. We additionally introduce a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector; by concatenating M_{global} with the purified node sequence embedding and training them jointly,

Table 5: Model accuracy under the unified imperceptible attack, with percentage decrease from clean accuracy reported in parentheses.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

the model learns to process potentially perturbed graph inputs more robustly. Because GALGUARD is designed as a plug-and-play framework, alternative defenses, e.g. Pro-GNN (Jin et al. 2020), can replace GNNGuard when appropriate.

10.1 Results

Table 6 reports model accuracy under METAATTACK together with two defense variants: GALGUARD_p, which combines feature correction with graph purification, and the full GALGUARD, which further incorporates GNNGuard or its LLAGA-specific adaptation. Across Cora, CiteSeer, and PubMed, and under both poisoning and evasion attacks, the proposed defenses consistently attenuate the effect of METAATTACK. In particular, GALGUARD_p provides an effective intermediate defense, showing that LLM-based feature correction coupled with initial graph purification materially improves robustness. The full GALGUARD, however, consistently delivers stronger robustness gains than GALGUARD_p, indicating that the GNN-inspired structural defense contributes synergistically rather than redundantly.

Table 6: Accuracy under METAATTACK and the proposed GALGUARD variants for poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper provides an initial systematic characterization of adversarial vulnerabilities in graph-aware LLMs, a setting that has remained largely unexplored despite the increasing

adoption of LLM-based graph models. Our findings show that the mechanisms used to integrate graph data into LLMs constitute a critical source of vulnerability. In particular, we identify new attack surfaces, including the node sequence template in LLaGA, and demonstrate that such architectural design choices can substantially increase adversarial susceptibility. We further show that, unlike in many traditional GNN settings, imperceptible feature perturbation attacks are highly effective against graph-aware LLMs, often exceeding the impact of structural attacks; when feature and structural perturbations are combined in a unified attack, model utility can be severely degraded.

To mitigate these risks, we propose GALGUARD, an end-to-end defense that integrates an LLM-based feature corrector with adapted GNN-based structural defenses. As graph-aware LLMs become increasingly prevalent, with potential applications across social networks, healthcare, and finance, their resilience to adversarial exploitation becomes essential. We hope that this work establishes a foundation for further research on the vulnerabilities and defenses of graph-aware LLMs, motivating both deeper theoretical analysis and practical safeguards for real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.

- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure-text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.

- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 provides a detailed taxonomy of graph-aware LLMs, and Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 summarizes the adversarial mechanisms of NETTACK and METAATTACK, whereas Appendix A.4 specifies the algorithm for the node sequence injection attack on LLaGA. Appendix A.5 reports ablation results, including the effects of perturbation magnitude and base-LLM choice on attack performance. Finally, Appendix A.7 provides feature-attack examples and the LLM-corrector template, and Appendix A.8 discusses broader implications of the study.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data has emerged as an active research area, with numerous approaches examining how LLMs can be adapted or augmented for graph reasoning, representation, and inference. These approaches can be broadly divided according to how graphs are encoded for LLM consumption: *textual descriptions of graphs*, in which graph information is rendered as natural language, and *learned graph projector* methods, in which a projector maps graph data into embeddings or structured representations that LLMs can process directly. This section describes the principal methods within each encoding family. Figure 5 illustrates both approaches, and Table 7 summarizes their main differences.

Graph Representation as a Textual Description In the first category, graph data is represented as natural language, enabling LLMs to process graph structures through textual descriptions rather than specialized graph encoders. This strategy has been extensively explored in work that converts graph data into textual formats, often paired with task-specific querying instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a). For example, Tan et al. (2023) introduce a unified framework for attributed graph embedding by generating random-walk-based textual sequences that are subsequently used to fine-tune LLMs. Similarly, Zhao et al. (2023) study graph reasoning within textual spaces by translating graph structures into natural language to facilitate LLM interaction. Extending this direction, Wei et al. (2024a) augment textual graph representations with visual graph renderings, thereby improving LLM reasoning over graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) link molecular graphs with textual descriptions, supporting advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) examine whether incorporating graph structure into LLM prompts improves predictive performance, finding that LLMs often process graph-related prompts as contextual paragraphs rather than explicitly recognizing graph topology. This observation reinforces the importance of prompt design, as reflected in studies such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023;

Bi et al. 2024; Pan et al. 2024), which extend LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) investigate the use of LLM-generated explanations as features that enhance both performance and interpretability on graph tasks.

A related line of work, while still retaining textual graph representations, integrates LLM pretraining strategies with GNNs in order to strengthen graph reasoning. For instance, Chen et al. (2023a) propose two LLM-GNN integration approaches: using LLMs to enhance node features for node classification and employing LLMs as direct predictors for graph tasks. Along similar lines, Chandra et al. (2020) use LLMs as text encoders and GNNs as graph-structure encoders for tasks such as fake-news detection in online communities. Chen et al. (2023b) explore node classification without explicit labels by using LLMs to annotate nodes that are subsequently used to train a GNN, while Mavromatis et al. (2023) use an LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) propose a textual-graph fine-tuning method that combines LLMs and GNNs through a tunable side structure attached to each LLM layer, improving both training efficiency and inference speed. Similarly, Duan et al. (2023) introduce SimTeG, a simple yet effective approach for textual graph learning in which the LLM is first fine-tuned on a downstream task such as node classification, after which node embeddings are obtained from the final hidden states of the fine-tuned LLM. Additionally, Zhang (2023) propose Graph-toolformer, which uses ChatGPT-assisted prompt engineering to augment LLM graph reasoning.

Although these approaches make LLMs more accessible for graph-related tasks, they often struggle to capture the structural intricacy inherent in graph data. Their performance also remains highly dependent on prompt engineering, a labor-intensive process that requires careful design and repeated experimentation to obtain strong results.

Graph Representation as a Learned Projector To address the limitations of textual representations, learned-projector methods transform graph data into embeddings or graph vectors that LLMs can process more directly. For instance, Chen et al. (2024) introduce LLaGA, a large language and graph assistant that preserves the general-purpose character of LLMs while converting graph structures into graph tokens suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporate graph-structure information into a pre-trained LLM through tailored disentangled graph neural network layers. Tian et al. (2024) propose graph neural prompts for encoding knowledge graphs into graph embeddings that LLMs can use during inference. Furthermore, Huang et al. (2023b) introduce a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs, combining a graph adapter, implemented as a learnable GNN layer, with task-specific prompts to capture neighborhood information and extract informative node features.

Another promising direction compresses graph information into fixed-length prefixes with graph transformers that are attached to each LLM layer to enhance reasoning (Chai et al. 2023). Similarly, Yang et al. (2021) develop GraphForm-

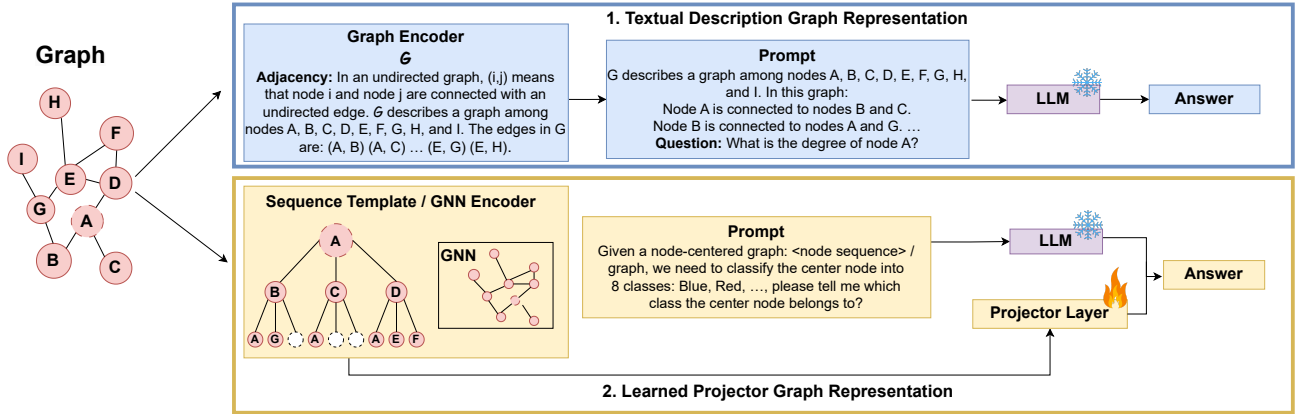


Figure 5: **Graph encoding-based adaptations of LLMs.** Under the textual-description representation, graphs are verbalized as natural-language text (top). Under the learned-projector representation, graphs are encoded either by node templates that convert topology into sequences or by GNN-based encoders (bottom). In both cases, the LLM remains frozen, and only the projector mapping graph information into token space is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

ers, which integrates text encoding and graph aggregation through an iterative workflow that nests GNNs alongside the transformer blocks of a language model. Additionally, Peng et al. (2024) introduce ChatGraph, an interactive interface through which users query graphs in natural language; in this setting, graphs are represented as sequential paths, and the LLM is fine-tuned on graph data to support graph analysis.

In this paper, we exclude projector adaptations that require LLM retraining, because they substantially increase computational cost without commensurate performance gains. Instead, we focus on adaptation methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This design reduces computational cost, improves compatibility with different LLM backbones, and preserves access to the broad knowledge encoded in pre-trained LLMs while still enabling task-specific graph adaptation. Throughout the work, we use two representative projector methods: **LLAGA** (Chen et al. 2024), which relies on node-level templates and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector to encode the graph.

This work. In this work, we do not consider textual-description methods because they often fail to capture the structural intricacies of graph data with sufficient fidelity. Such methods are generally most suitable for simpler tasks, such as describing basic graph properties like node degree, and become increasingly infeasible for larger graphs because prompt token limits restrict the level of structural detail that can be represented. From a vulnerability perspective, textual-description methods also do not introduce a distinctive graph-specific attack surface: because the graph is expressed as ordinary text, e.g., "Node A is connected to Node B", these methods are primarily exposed to conventional text-based adversarial attacks rather than to the structural or feature perturbations specific to graph-aware model architectures. They are therefore less central to our investigation of adversarial robustness in graph-aware LLMs.

A.2 Attacks on GNN

GNNs are vulnerable to several attack families, including membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a specific node was included in training; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit model outputs or learned embeddings to infer sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract global graph properties; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance through subtle graph or feature perturbations; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which reconstruct graph structure from learned embeddings or model outputs.

Among these categories, adversarial attacks are especially consequential because they directly target predictive performance. Such attacks introduce small, carefully crafted *perturbations* into the graph, for example by modifying node

features or altering edges, with the aim of substantially degrading GNN performance. They are typically divided into **poisoning attacks**, which manipulate training data during the *training phase*, and **evasion attacks**, which perturb inputs at *test time* to mislead predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) show that perturbations to graph structure or node features can substantially degrade GNN performance; their method, **NETTACK**, uses greedy optimization to find minimal perturbations that maximize training loss and thereby identifies small feature or structural changes capable of disrupting predictions. Li et al. (2023) revisit graph adversarial attacks from a data-distribution perspective, showing that adversarial examples can substantially alter the underlying distribution and proposing a defense to mitigate this effect. Similarly, Wu et al. (2019) provide a comprehensive analysis of graph adversarial examples, examining GNN sensitivity to small perturbations and proposing defenses that detect and counteract adversarial modifications. **METAATTACK** (Zügner and Günnemann 2019) performs both poisoning and evasion attacks by treating graph structure as an optimizable hyperparameter; it modifies graph topology through gradient descent in a bi-level optimization formulation, using meta-gradients that capture how structural perturbations affect the attacker’s post-training loss and thereby enable precise graph updates that degrade performance.

In this paper, we use **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial methods spanning both poisoning and evasion. Poisoning attacks target model integrity during training, whereas evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding and removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization, with respect to a surrogate model, to identify minimal perturbations that maximize the model’s training loss. The surrogate is a linearized 2-layer graph convolutional network (GCN) that approximates how structural changes affect classification outcomes. The primary objective of **NETTACK** is to increase confidence in incorrect classifications by maximizing the distance between logits under the perturbed and clean graphs. To do so, **NETTACK** enumerates candidate edge perturbations around a target node and evaluates a score function for each connected edge, where the score estimates how strongly changing that edge will affect the target node’s classification. The highest-scoring edge is then added or removed. Because **NETTACK** is a targeted attack, causing misclassification of a specific node, we follow (Zügner and Günnemann 2019) to convert it into an untargeted evaluation that compromises aggregate node-classification performance: one test node is randomly selected and attacked with **NETTACK** while all network nodes are considered, and the results are then aggregated over test nodes.

METAATTACK optimizes graph structure directly through gradient descent by treating the adjacency matrix, i.e., the graph structure, as a hyperparameter. It formulates the attack as a bi-level optimization problem: the inner level minimizes

training loss on the original graph, while the outer level optimizes the poisoned graph objective. To approximate the effect of structural changes on model performance, METAATTACK uses a surrogate linearized two-layer GCN. A score function evaluates the impact of each candidate perturbation, and the attack flips the sign of meta-gradients, i.e., gradients with respect to hyperparameters, for connected nodes to identify edges for removal. The algorithm then greedily selects the highest-scoring perturbations, adding or removing edges so as to maximize attack impact.

For both methods, we impose the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which restricts changes to the graph’s degree distribution; accordingly, only 10% of edges are perturbed.

In poisoning experiments with LLAGA, we use the perturbed graph produced by NETTACK or METAATTACK to generate node-level sequences and then retrain the projector that converts those sequences into token embeddings. For GRAPH-PROMPTER, we similarly use the perturbed graph, re-encode it with a graph attention network (GAT), and retrain the projector on the perturbed structure. In evasion experiments, the graph-aware LLM is trained on unperturbed data, and perturbations are introduced only during inference.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 details the node sequence template injection attack for LLAGA. The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders are inserted, and these placeholder positions create the relevant attack surface. We exploit this mechanism by manipulating the graph so that malicious nodes occupy placeholder slots. The attack comprises three strategies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts a high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). By introducing uninformative nodes into the sequence, the attack corrupts the target representation and degrades projection-layer training and performance.

A.5 Ablation Studies

This section provides the supplementary ablation results referenced in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 reports the performance of LLAGA and GRAPH-PROMPTER as the percentage of perturbed features varies. Increasing the proportion of perturbed features consistently reduces accuracy for both models, confirming their sensitivity to feature-level attacks.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 indicate that the base LLM has minimal influence on both clean performance and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B- and Llama2-7B-based variants achieve nearly identical clean accuracy

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k
Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$ 
       neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

and exhibit comparable degradation under poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, performance falls to 0.44 ± 0.06 for Vicuna-7B and

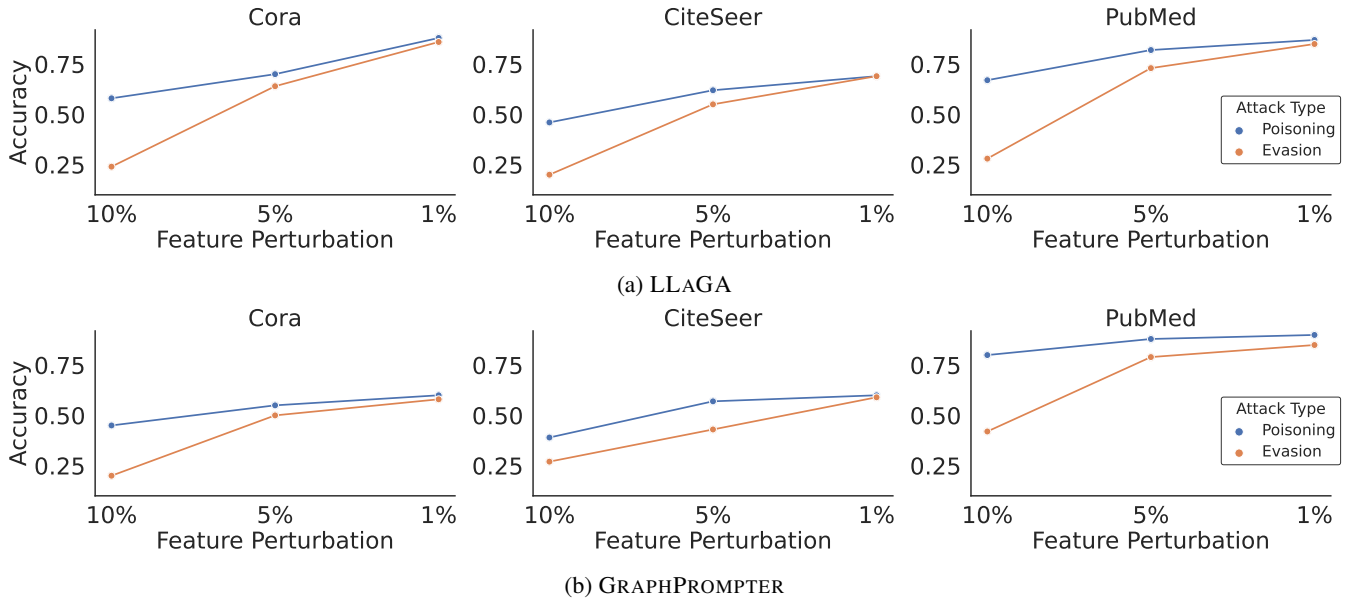


Figure 6: Accuracy of graph-aware LLMs as the percentage of perturbed features varies; lower accuracy indicates a more successful attack.

0.43 ± 0.08 for Llama2-7B, a difference well within the reported standard deviation. Analogous patterns hold across datasets and attack types, suggesting that attack effectiveness is largely independent of the base LLM.

A.6 Results on the ArXiv Dataset

We next evaluate the attack on the large-scale ArXiv dataset, which contains 169,343 nodes and 1,166,243 edges, with sparsity 0.81. This experiment examines whether node sequence template attacks remain effective in a large, real-world graph. As shown in Table 9, poisoning and evasion attacks exhibit a substantial performance gap: poisoning consistently produces stronger degradation than evasion across all three injection strategies, a pattern not observed on Cora, CiteSeer, or PubMed. This suggests that ArXiv may possess structural characteristics that make it more vulnerable to poisoning while comparatively more robust to evasion. Although ArXiv is highly sparse, the insertion of even a single supernode can still substantially affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 illustrate our adversarial attacks on real Cora node features. The “input” corresponds to the text received by the LLM at the point where the attack is applied. Although the original and perturbed strings appear visually identical, or nearly identical, to a human reader, the LLM processes different Unicode code points or a different character order, which can alter its behavior. Table 10 illustrates this phenomenon for the Homoglyph attack, and Table 11 does so for the Reordering attack.

Feature Corrector Defense Template We design a feature-corrector module to mitigate imperceptible attacks, specifically homoglyph substitutions and character reorderings. The module is implemented as a prompt-based LLM defense, with the prompt instructing the model to act as a linguistic validator that detects and corrects subtle perturbations that are difficult for humans to perceive.

The prompt template is shown below. It explicitly defines the task and the required output format, which improves the reliability of the LLM’s corrections.

A.8 Detailed Discussion

The findings of our work raise several critical insights into the vulnerabilities and design considerations of graph-aware LLMs. As the integration of graph-structured data into LLMs continues to gain traction for graph tasks, it is important to explore not only the performance benefits but also the vulnerabilities of these novel architectures. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs, LLAGA and GRAPHPROMPTER —reveals significant weaknesses, particularly when exposed to poisoning and evasion attacks, which are traditionally used to undermine GNNs. Our key observations are discussed below.

Impact of Graph Encoding on Adversarial Vulnerability The key observation from our results is that the integration method of graph structure into LLMs can inadvertently increase their susceptibility to adversarial attacks. Specifically, the node sequence template employed in LLAGA appears to create a vulnerability that can be exploited by adversaries, leading to substantial degradation in model performance. This attack surface, unique to LLAGA, highlights a critical tradeoff in graph-aware LLM design: while the use of node-level templates provides an effective means of incor-

Table 8: Effect of base-model substitution on LLAGA’s clean accuracy and adversarial performance under different attacks. The results indicate that base-model choice does not substantially alter attack susceptibility.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Prompt for Feature Corrector	
You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged. Example 1 (Homoglyph Substitution): Input: Title: The mgprir huristic fr discvrng prtln squnc pttrens Abstract: Svrl cmputr lgrithms fr discvrng pttrens in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f sttistical mdl to grup of rlted squncs. Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences. Example 2 (Character Reordering): Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the apclpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae. Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made. Example 3 (Mixed Homoglyph & Reordering): Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In clulur tlphne systms, n imrtnt prblm is to dynmicly alloct th cmmunicin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblm is naturally frmultd s dynmic prgmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl lliction plicies that ar bttr than prvius heuristic slutins. Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions. Your Task: Text to correct: {perturbed_text_here} Corrected Text:	

Figure 7: Prompt template for the LLM-based feature corrector module, explicitly specifying the LLM’s role, task, and expected output format.

Table 9: Accuracy under node sequence template injection attacks across datasets and attack variants. *NI*, *SI*, and *MSI* denote non-adjacent injection, single-supernode injection, and multiple-supernode injection, respectively, while Clean denotes model performance without attack.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 $\pm$ 0.07	————
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	————	0.71 $\pm$ 0.04	————
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	————	0.90 $\pm$ 0.04	————
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03
ArXiv	Clean	————	0.73 $\pm$ 0.02	————
	Poisoning	0.50 $\pm$ 0.03	0.43 $\pm$ 0.04	0.42 $\pm$ 0.03
	Evasion	0.71 $\pm$ 0.06	0.69 $\pm$ 0.02	0.60 $\pm$ 0.03

porating graph structure and achieves better performance on graph task, it also exposes the model to new attack vectors not present in traditional GNNs. On the other hand, GRAPH-PROMPTER, which uses a GNN-based projector for graph encoding, demonstrates comparatively greater resilience to these attacks, underscoring the potential advantages of hybrid architectures that leverage the strengths of both LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations In addition to the vulnerabilities in the graph encoding techniques, our results also highlight the effectiveness of feature perturbation attacks across both graph-aware LLMs. Despite differences in how graph structure is encoded, both LLAGA and GRAPH-PROMPTER exhibit significant sensitivity to imperceptible perturbations in the node features. This highlights a broader trend observed in the LLM paradigm: while these models excel in handling textual data, their robustness to adversarial perturbations, especially those targeting node features, remains a critical challenge. The ability of adversaries to degrade model performance by subtly altering node attributes without being detected calls into question the integrity of graph-aware LLMs, especially in security-sensitive applications. While text sanitization techniques, such as those discussed by (Boucher et al. 2022), offer potential defenses, addressing these imperceptible attacks such as homoglyph-based manipulations remains a complex challenge. As proposed, using LLMs as a corrector is one way to mitigate such vulnerabilities, by prompting the model to detect and repair suspicious or malformed inputs before downstream processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings reveal that the efficacy of poisoning and evasion attacks in the context of graph-aware LLMs is not uniform across datasets. While attacks are particularly effective on the Cora, Citeseer and PubMed datasets, we observe a more nuanced interaction

between the attack strategies and the graph structure on the ArXiv dataset. This variation emphasizes the importance of dataset characteristics in evaluating the vulnerabilities of graph-aware LLMs. As these models are increasingly deployed in diverse real-world settings, their vulnerabilities must be assessed not only in terms of algorithmic design but also in relation to the specific domains and data they are applied to.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study have broader implications for the future development and deployment of graph-aware LLMs. Given their increasing adoption for tasks that were traditionally the domain of GNNs, it is crucial to consider the tradeoffs between performance and vulnerabilities in the design of these paradigms. As graph-aware LLMs continue to evolve, there is a pressing need for the development of more robust encoding techniques and defenses against adversarial manipulation, particularly in the context of feature perturbations. Furthermore, understanding how adversarial attacks transfer from GNNs to graph-aware LLMs opens new avenues for research in protecting these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have predominantly emphasized structural perturbations, because feature-based perturbations typically fail to change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and therefore have limited practical effectiveness. Consequently, there are *no known defenses against feature-based attacks on GNNs*. As demonstrated in the main paper, however, feature perturbations become effective in the graph-aware LLM paradigm, creating a new challenge in which attacks that were previously weak against GNNs can now manipulate model behavior. This shift motivates defenses that jointly address structural and feature-level adversarial perturbations. Although text sanitization methods, including optical character recognition (OCR) as a preprocessing step before LLM encoding, may appear promising, imperceptible attacks such as homoglyph manipulation remain difficult because OCR is imperfect and tends to misinterpret homoglyphs more frequently than unperturbed text (Boucher et al. 2022). This limitation motivates our LLM-based self-correction mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs, particularly in evasion settings, it may seem natural to assume that GNN defenses will transfer as well. This assumption, however, does not hold: unlike traditional GNNs, *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective*.

One GNN defense mechanism makes the adjacency matrix trainable, allowing edge weights to adapt during training in

Table 10: Illustration of the imperceptible Homoglyph attack. Although the LLM processes the perturbed text as distinct characters, the rendered output remains nearly identical to a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc ptttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible Reordering attack. Although the LLM processes the perturbed text as a different character sequence, the rendered output remains nearly identical to a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identfy risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

order to mitigate adversarial perturbations. This approach assigns lower weights to edges connecting dissimilar nodes and higher weights to edges linking nodes with greater feature similarity. Several **preprocessing-based** defenses build on this principle, for example by removing edges between nodes with low feature similarity (Wu et al. 2019) or by applying randomized edge dropping (Dai et al. 2018). These defenses, however, rely heavily on clean node features to compute reliable similarity scores. When node features are also perturbed, as in our setting, the similarity estimates become unreliable, and the defenses may even exacerbate the problem by reconstructing incorrect edges. We demonstrate this effect for a random node in Cora (Figure 8): while the clean feature-similarity graph preserves some original edges while introducing some erroneous ones, the graph reconstructed from perturbed features preserves no original connections,

yielding an entirely incorrect reconstruction.

Another defense direction reduces reliance on graph structure. For instance, (Shanthamallu, Thiagarajan, and Spanias 2021) propose training a surrogate predictor that depends only on node features and uses uncertainty matching to extract graph information from the GNN. Similarly, Wu et al. (2021) introduce a co-training framework in which submodels are trained independently on feature and structural views, with knowledge distilled by adding confident unlabeled data to the training set. These approaches, however, also assume access to unperturbed node features, an assumption violated in our setting where both structure and features are adversarially modified. Consequently, structural defenses developed for GNNs are not directly applicable to graph-aware LLMs, motivating defenses tailored to this paradigm.

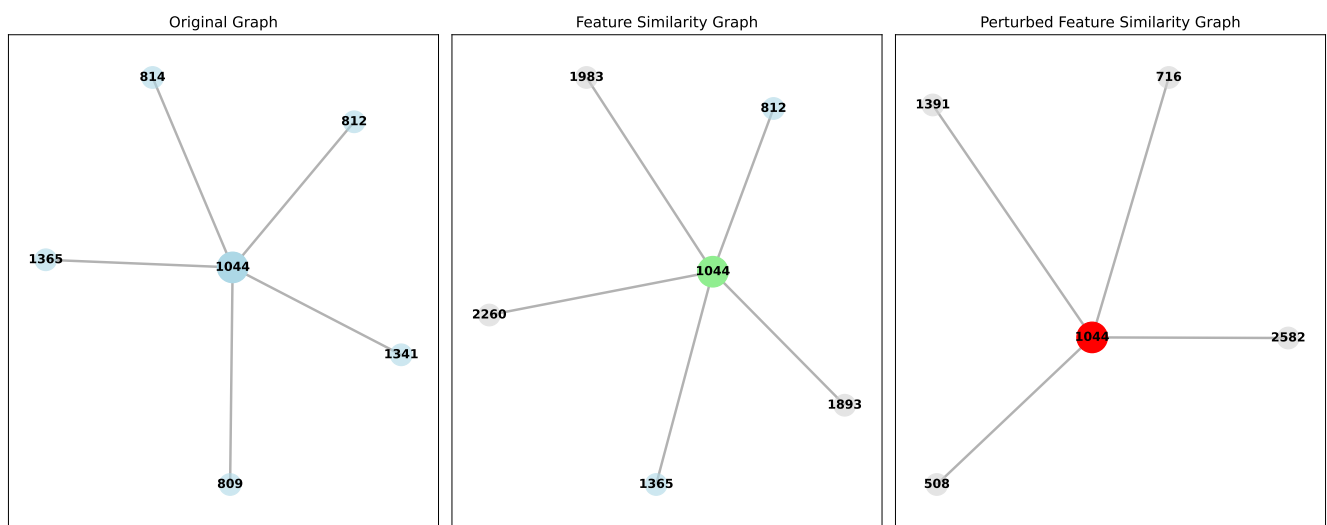


Figure 8: Edges reconstructed for random node 1044. **Original (left)** shows ground-truth edges from the original graph. **Feature Similarity (middle)** shows edges reconstructed from clean node features, with blue nodes indicating preserved original connections. **Perturbed Feature Similarity (right)** shows edges reconstructed from perturbed node features.